



Session 4: Agile, Scrum & Enterprise Engineering Workflow

From Business Requirement to Production Value

Agile Mindset

Iterative, value-driven
delivery

Scrum Framework

Sprints, ceremonies, artifacts

Enterprise Governance

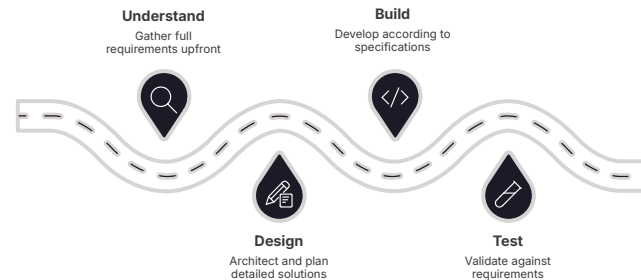
Control, traceability, compliance

Why Do We Need Agile?

What Changes?

- Requirements change
- Regulations change
- Customer expectations shift
- Technology evolves

Traditional Approach



Problems surface **late** — after months of work.

- ❑ Agile replaces one big release with continuous, incremental delivery.

What Is Agile?

A **mindset and delivery philosophy** — build incrementally, incorporate feedback continuously.



Customer Collaboration

Working closely with stakeholders throughout delivery



Frequent Delivery

Usable software shipped in short cycles



Continuous Feedback

Validate early, adapt often



Technical Quality

Sustainable pace, clean code, continuous improvement

📌 **Agile ≠ Scrum.** Agile is the philosophy. Scrum is one framework to implement it.

The Agile Manifesto

Individuals & Interactions

over processes and tools

Working Software

over comprehensive documentation

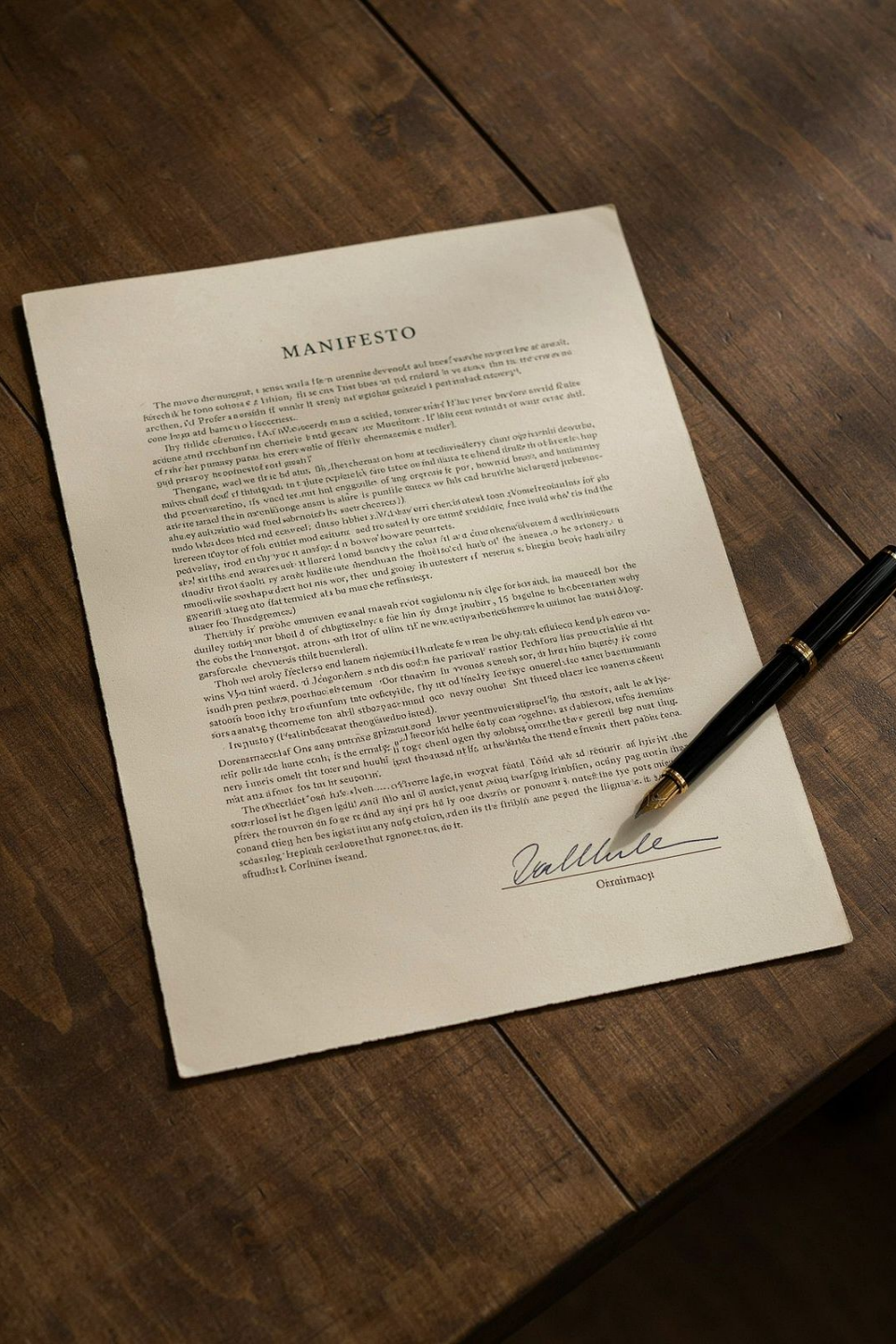
Customer Collaboration

over contract negotiation

Responding to Change

over following a plan

The right-side items still matter — Agile places **greater value on the left**.



Traditional vs. Agile Delivery

Traditional	Agile
Large upfront planning	Progressive planning
Requirements fixed early	Requirements evolve
Large releases	Incremental releases
Testing near the end	Continuous testing
Feedback arrives late	Frequent feedback
Change is expensive	Change is expected

Mortgage Example

01

Sprint 1

Borrower onboarding

02

Sprint 2

Document verification

03

Sprint 3

Underwriting workflow

04

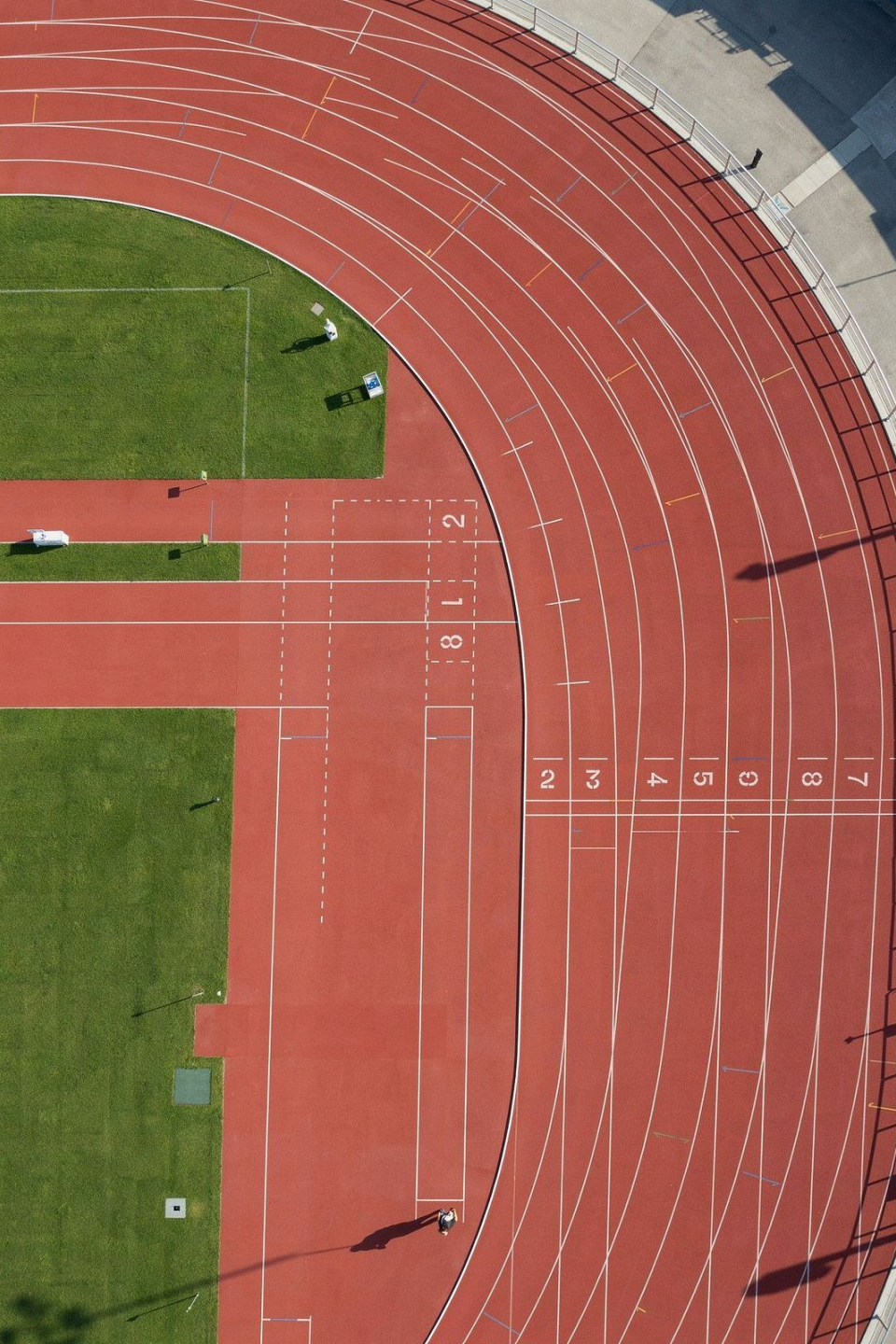
Sprint 4

Servicing

05

Sprint 5

Delinquency alerts



What Is Scrum?

A lightweight framework for solving complex problems through **short, iterative cycles called Sprints**.

1-4

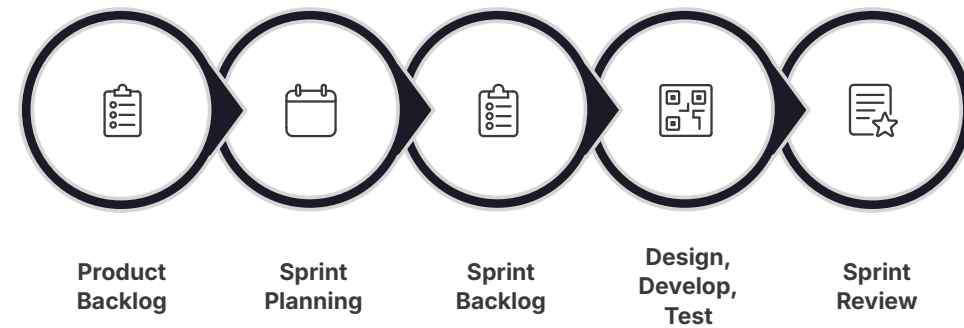
Week Sprint

Typical range

2

Weeks Common

Enterprise standard



Scrum Roles & Accountabilities



Product Owner

- Defines priorities
- Manages Product Backlog
- Clarifies requirements
- Accepts business functionality



Scrum Master

- Removes impediments
- Facilitates ceremonies
- Coaches the team
- Protects Agile practices



Delivery Team

- Developers, QA, Security
- Data, DevOps, UX
- Creates the increment
- Cross-functional ownership

Cross-Functional Delivery Model



One team. One outcome.

Business

Product Owner · Business Analyst

Engineering

Java/Python · UI · Data Engineers

Quality

QA · Automation Engineers

Platform

DevOps · Cloud · SRE

Security

Security Engineers

- ☐ The team owns delivery from requirement to production — not handoffs between silos.

Scrum Artifacts



Product Backlog

Ordered list of everything the product might need



Sprint Backlog

Work selected for the current Sprint plus the delivery plan



Increment

Usable product capability delivered at Sprint end

Sprint 1 selects only the first few items from a backlog that may include borrower registration, identity verification, mortgage application, underwriting, and more.

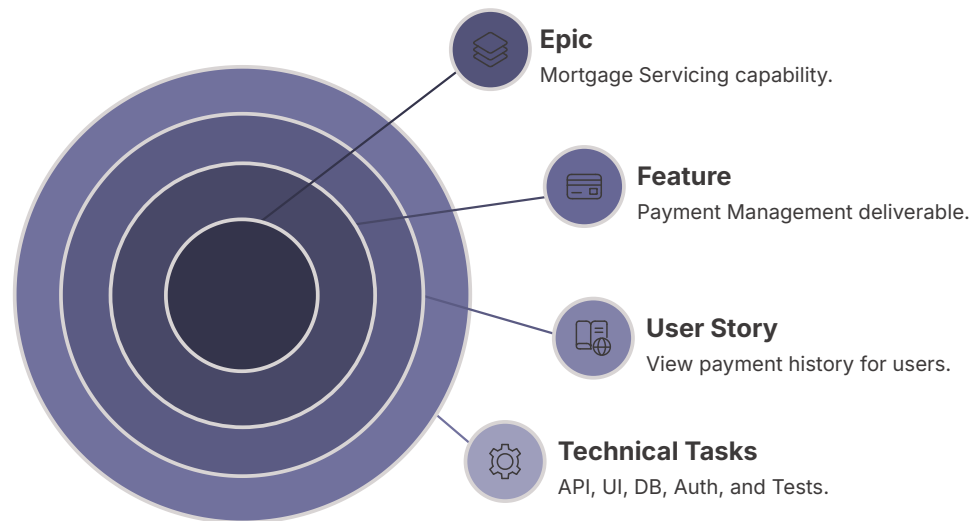
Understanding the Product Backlog

A **living, continuously evolving** list of product work — not a static requirements document.

- **Features & User Stories**
- **Technical & Architecture Work**
- **Security & Compliance Changes**
- **Defects & Performance Improvements**
- **Research / Spikes**

📌 The backlog is **ordered**, not merely stored. High-value items have the most detail.

Epic → Feature → Story → Task



Why decompose?

Turns **strategic requirements** into executable engineering work that fits inside a Sprint.

01

Epic

Large business capability

03

User Story

User-level requirement

02

Feature

Meaningful deliverable capability

04

Task

Concrete implementation activity



What Is a User Story?

As a [persona], **I want** [capability], **so that** [business value].

WHO

The persona who needs value

WHAT

The capability requested

WHY

The business outcome delivered

Example: As a borrower, I want to upload my income documents so that my mortgage application can be verified.

User Stories Are Conversations

A story is not a mini requirements document — it is an **invitation to discuss**.

Card

Short description of the requirement

Conversation

Discussion between Product, Engineering, QA, Security

Confirmation

Acceptance criteria that verify expected behavior

THE 3 CS

INVEST: Characteristics of Good Stories



Independent

Deliverable with minimal dependency



Negotiable

Details evolve through discussion



Valuable

Delivers identifiable business value



Estimable

Team understands it well enough to size



Small

Deliverable within a single Sprint



Testable

Expected behavior can be verified

From Vague Requirement to Good Story

Vague Business Request

"We need better delinquency management."

- **Who needs it?**
- **Which condition?**
- **What outcome?**
- **How do we verify it?**

Better Story

As a servicing specialist, I want loans exceeding the configured delinquency threshold to be automatically flagged so that I can prioritize borrower follow-up.

Now engineering can discuss: threshold, data source, frequency, notifications, authorization, error handling, and audit trail.

What Are Acceptance Criteria?

Conditions that must be satisfied for the story to meet expected business behavior. Creates shared understanding across **Product + Dev + QA + Security**.

Story

As a borrower, I want to upload income documents.

Acceptance Criteria

- **Supported file types defined**
- **Maximum file size enforced**
- **Unauthorized users blocked**
- **Successful upload creates confirmation**
- **Failed upload returns meaningful error**
- **Upload action is auditable**

Given / When / Then

A behavior-oriented format that aligns business requirements with automated testing.

Given

Initial condition or context

When

Action or triggering event

Then

Expected outcome or result

Given an authenticated borrower **And** a valid mortgage application exists

When the borrower uploads a supported document

Then the document is securely stored **And** associated with the application **And** an audit event is recorded.

Definition of Ready



Checks before entering a Sprint

- **Business objective understood**
- **Acceptance criteria defined**
- **Dependencies identified**
- **UX/API expectations understood**
- **Security requirements identified**
- **Story is small enough to estimate**

□ DoR is a team practice, not a mandatory Scrum artifact. Avoid pulling ambiguous work into active Sprints.

Definition of Done

Code Complete $\neq$ Done

Engineering

- Code completed
- Peer review completed
- Unit tests passed
- Integration tests passed

Security & Quality

- Security scans passed
- No critical vulnerabilities
- Acceptance criteria satisfied

Operations

- Documentation updated
- Logging added
- Build successful

What Is Sprint Planning?

The event that starts every Sprint. The team aligns on three questions:

Why?

Why is this Sprint valuable?

What?

Which backlog items can be delivered?

How?

How will selected work be implemented?

Output: **Sprint Goal + Sprint Backlog + Delivery Plan**



Sprint Goal

✗ Weak Sprint Goal

Complete 14 Jira stories.

No clarity on purpose or outcome.

✓ Strong Sprint Goal

Enable borrowers to create mortgage applications and securely submit initial verification information.

If details change mid-Sprint, the team still understands the **intended outcome**.

📌 A Sprint should represent an **outcome**, not just a collection of tickets.

Backlog Refinement

Prepares upcoming work **before** Sprint Planning — not during it.



Split & Clarify

Break large stories, sharpen acceptance criteria



Design & Discuss

Architecture, security concerns, dependencies



Estimate & Order

Size stories, reorder backlog by priority



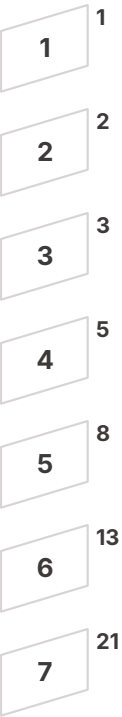
Good refinement makes Sprint Planning easier and more predictable.

Story Points

A relative estimation mechanism — not hours.

Complexity	Effort
Uncertainty	Risk

☐ 5 points ≠ 5 hours. Points reflect relative complexity, not time.



Common Fibonacci Sequence

Gaps widen as uncertainty grows — large stories are harder to estimate precisely.

Relative Estimation in Practice

Instead of asking *"exactly how many hours?"*, teams compare work to each other.

2 Points — Display borrower profile

Simple read, minimal logic, low
uncertainty

5 Points — Securely upload mortgage document

File handling, validation,
authorization, audit

8 Points — Integrate external identity verification

External dependency, failure
scenarios, security, high testing
effort

Planning Poker



Collaborative, bias-resistant estimation

01

PO explains story

02

Team discusses requirements

03

Each member picks a card privately

04

All reveal simultaneously

05

Outliers explain reasoning

06

Re-estimate if needed

- ❑ Simultaneous reveal prevents **anchoring** — the first voice shouldn't drive the estimate.

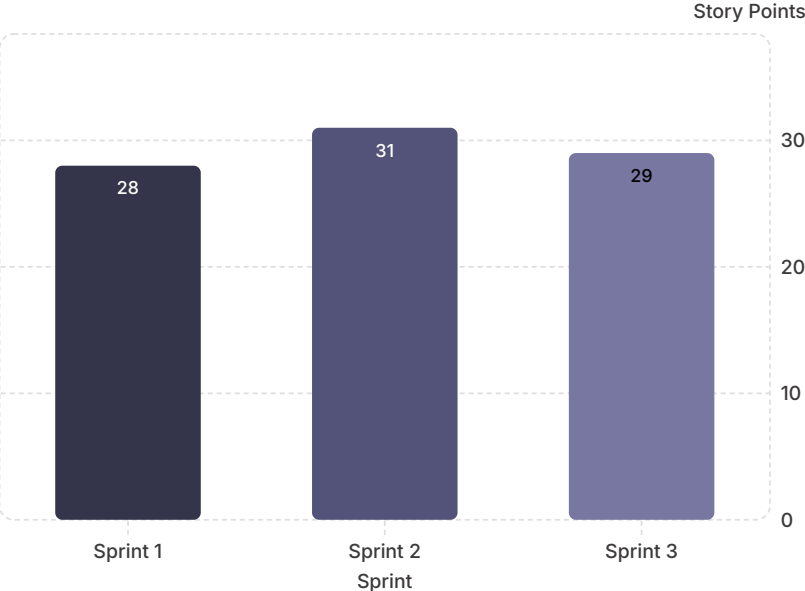
Velocity & Capacity

Capacity

Available team bandwidth this Sprint — affected by holidays, leave, training, and production support.

Velocity

Historical story points completed per Sprint.



Use velocity for planning, not judgment.

Velocity is a forecasting tool — not a performance metric for individuals.

Plan around the range

Not a single target number

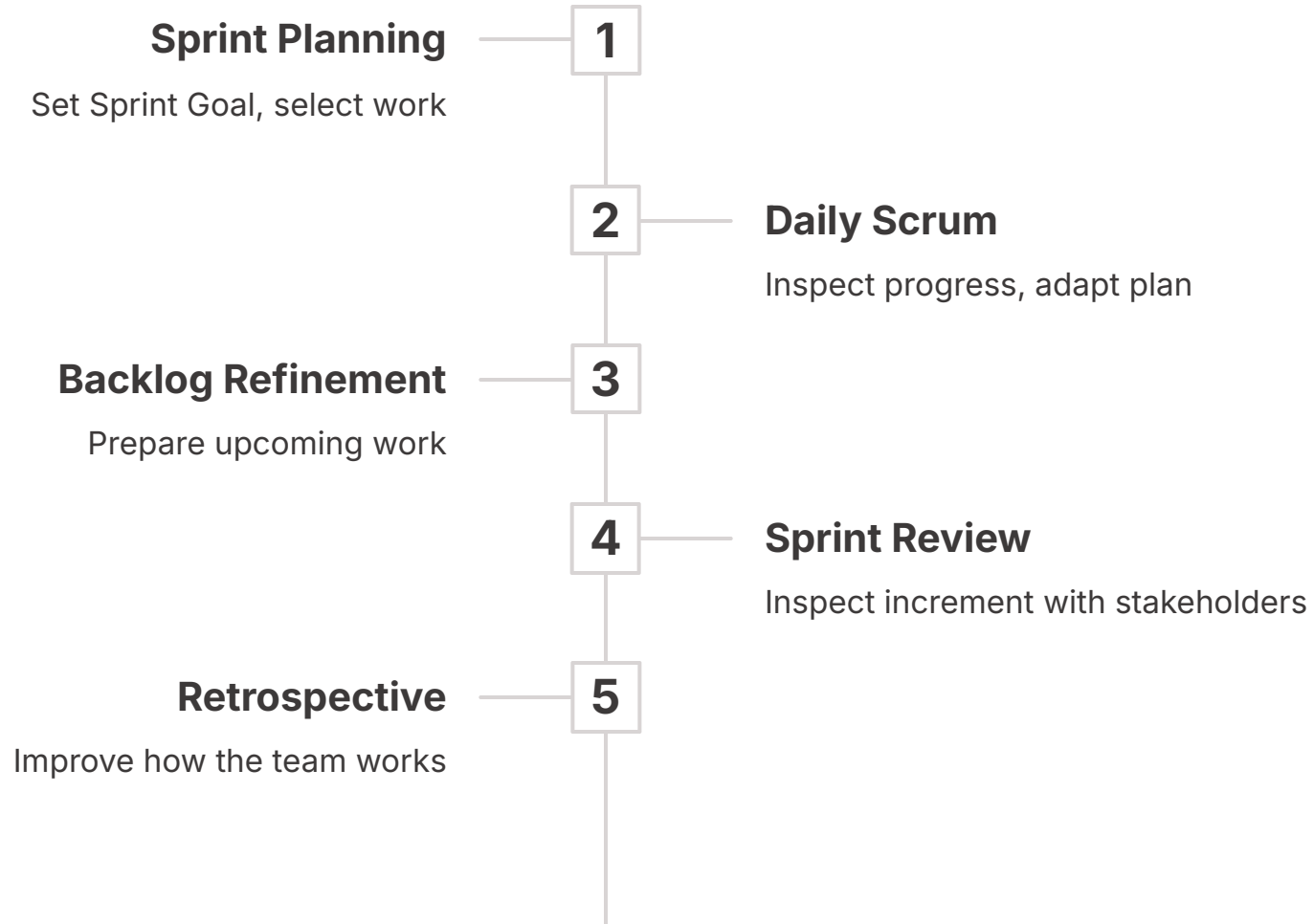
Adjust for capacity

Holidays and leave reduce availability

Never compare teams

Points are relative to each team

Scrum Ceremonies



Together they create a continuous **Plan → Execute → Inspect → Adapt** cycle.



Daily Scrum

15

Minute Timebox

Strictly enforced

✓ Should cover

- Progress toward Sprint Goal
- Current work and blockers
- Dependencies requiring coordination

✗ Should NOT become

- A lengthy technical deep-dive
- A management status meeting
- A problem-solving session

Detailed discussions can continue **immediately afterward** with relevant participants.

Sprint Review vs. Retrospective

Sprint Review

Focus: **Product**

- What was delivered?
- Does it meet stakeholder needs?
- What feedback was received?
- What should change in the backlog?

Improves the **product**.

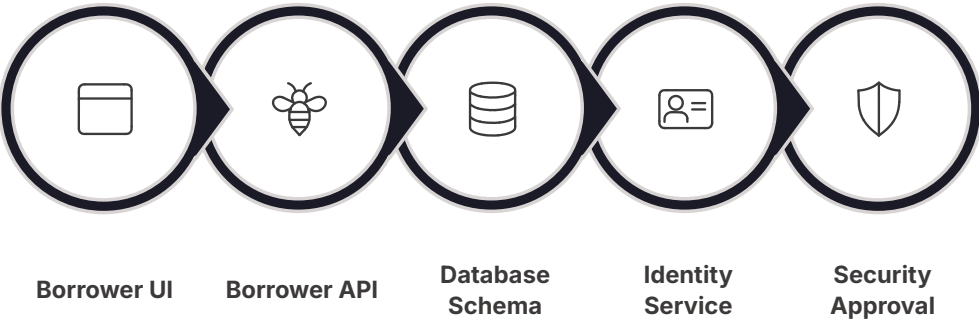
Retrospective

Focus: **Team & Process**

- What went well?
- What didn't work?
- What should we improve?
- What action will we take?

Improves the **delivery system**.

Technical Dependencies & Sprint Planning



Dependency Types to Identify Early

Technical

Team

Environment

Data

Security

Vendor

Governance

📌 Identify dependencies during refinement — not on the first day of the Sprint.

Enterprise Agile: Governance & Auditability

Agile does not eliminate governance — it must coexist with it in regulated environments.



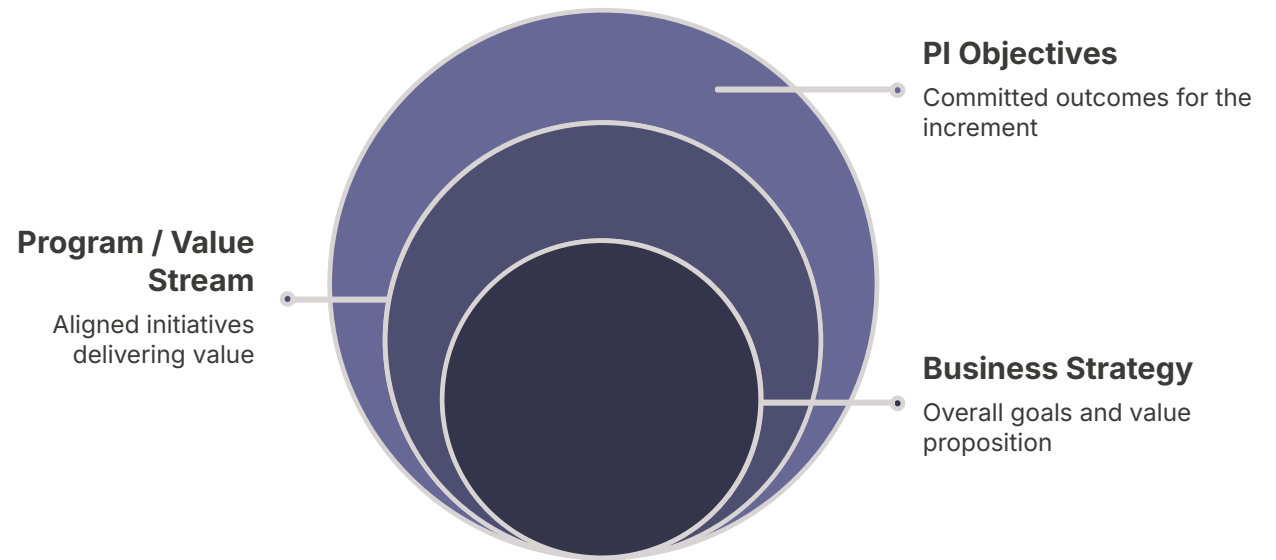
Maintain delivery speed while preserving **Control · Traceability · Compliance**.

PI Planning & Agile at Scale

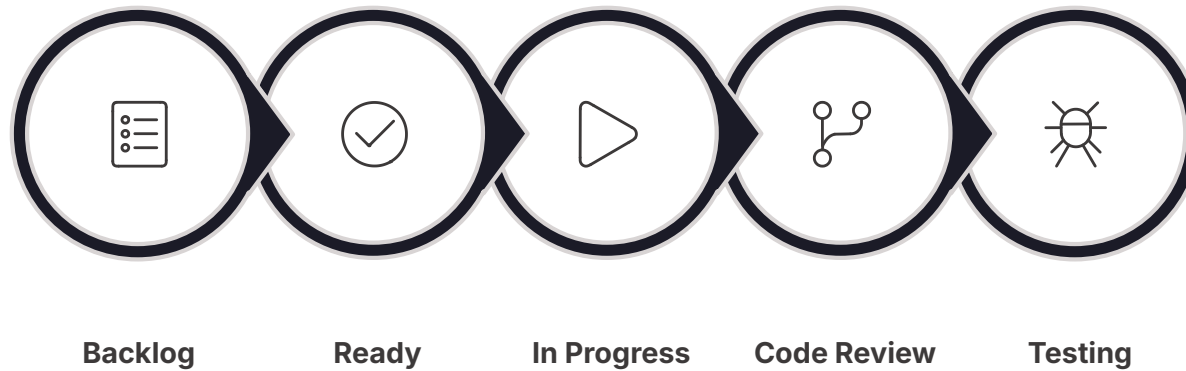


Program Increment (PI) Planning

Aligns multiple Agile teams around shared business objectives, priorities, features, dependencies, and delivery milestones.



Sprint Board Standards



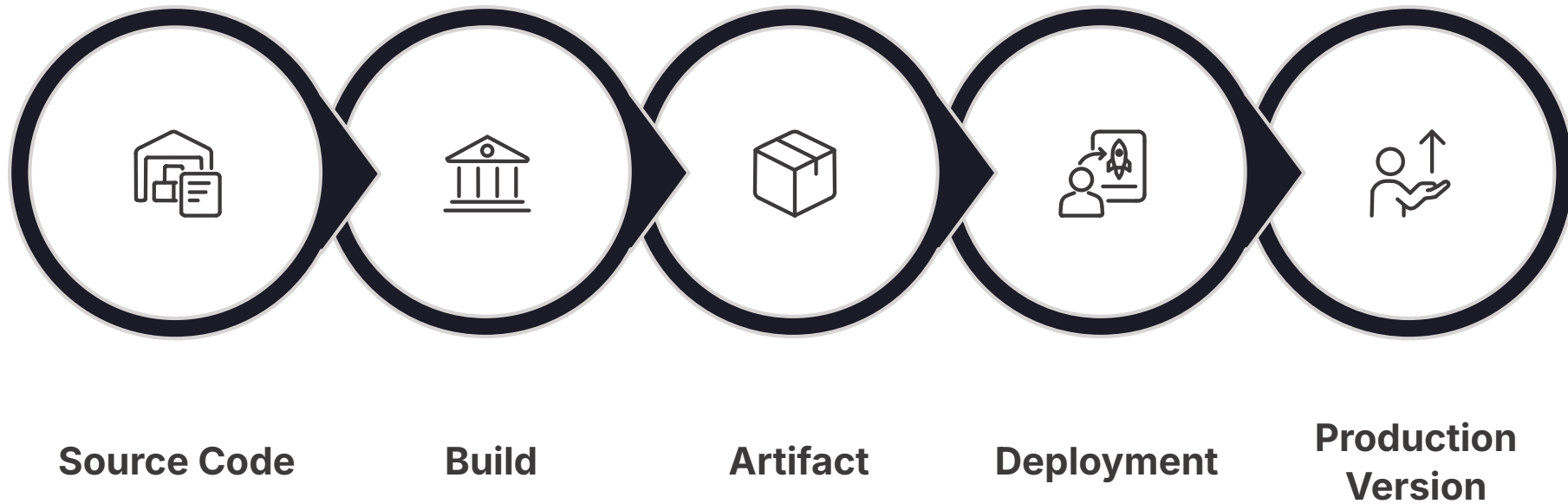
Enterprise Board Standards May Require

- Required fields & story ownership
- Acceptance criteria visible
- Labels, components, priority
- Dependency tracking
- Evidence links for compliance
- Definition of Done visible

☐ Anyone viewing the board should understand the **current delivery state** at a glance.

Release Tagging & Traceability

Not every Sprint produces an immediate production release — but every release must be fully traceable.



Rollback

Know exactly what to revert

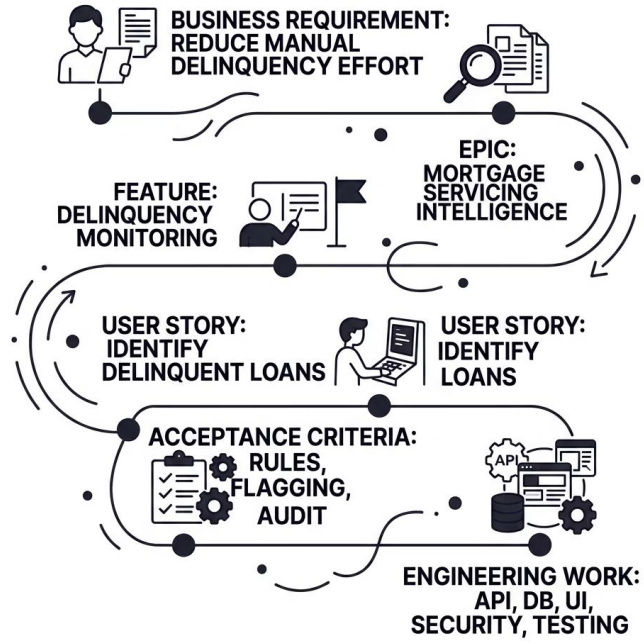
Incident Investigation

Pinpoint the change that caused the issue

Compliance & Audit

Demonstrate a full chain of evidence

Mortgage Domain: Requirement to Sprint



Acceptance Criteria

Engineering Work

API · Database · Rules Engine · UI · Security · Testing · Monitoring

- Delinquency rules are configurable
- Qualified loans are flagged automatically
- Authorized users can view results
- Borrower information is protected
- All actions are auditable

Key Takeaways



Agile is a mindset

Scrum is the delivery framework — not just a set of meetings



User stories drive conversation

Acceptance criteria create shared expectations across all disciplines



Story points measure complexity

Not hours — relative estimation reduces false precision



Cross-functional teams own outcomes

From requirement to production — no handoff silos



Governance and Agile coexist

Traceability, security, and compliance are built into the workflow



The full flow: Mortgage Need → Epic → Feature → Story → Acceptance Criteria → Estimation → Sprint → Dev → Test → Review → Release → Feedback → Backlog